RAFFLES INSTITUTION
H2 Computing (9569)

2025 Year 5

Chapter S1G: Hash Table

CONTENT

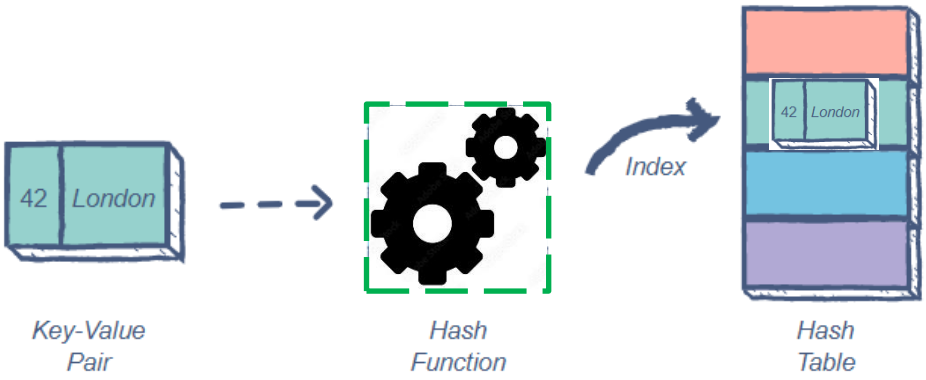
1. INTRODUCTION TO HASH TABLE

A hash table is a data structure that stores data as an **unordered collection of key-value pairs**. A **key** is processed by a **hash function**, which performs arithmetic operations to compute a **hash value**. This value determines the index (or address) in an array where the **key-value pair** will be stored. This method of indexing enables **fast searching**, **insertion** and **deletion operations**.

1.1 Key Components of a Hash Table

Hash Function

In hash tables, an element with key ***k*** is stored in slot ***h(k)***, where ***h*** is known as the **hash function**. The hash function determines the index of the key-value pair. Calculating an address from a key is called 'hashing'. Choosing an efficient hash function is a crucial part of creating a good hash table which performs effectively.



A good hash function should:

- Always produce the same hash value for the same key (deterministic).
- Distribute hash values uniformly to avoid clustering. This means that every hash value has ideally an equal probability of being generated.
- Minimise collision (when two keys hash to the same index).
- Be efficient to compute.

Array for Storage

The hash table uses an **array** to store data. Each **index** of the array corresponds to a **hash value** produced by the hash function. The **size of the array** should be chosen based on the **expected amount of data**. Since arrays allow direct access, data retrieval is very fast when the index is known.

When two different keys produce the same hash value, a **collision** occurs. Hash tables must use a **collision resolution technique**, such as **linear probing** or **separate chaining**, to handle these cases.

2. HASH FUNCTION

To insert data into a hash table, the **hash function** is used to generate an index in the array where the data will be stored. This index is calculated from the **key** associated with the data. Since the hash function does not store the original key itself, it is important that the **key is stored together with the data** in the hash table. This ensures that during retrieval or deletion, the correct data can be identified by comparing the stored key with the search key.

2.1 Modular Hashing

One common technique to compute an index from a numeric key is **modular hashing**. The hash function is defined as:

$h(k) = k \bmod m$ where k is the key, m is the size of hash table

The pseudocode for $h(k)$ is as follows:

```
FUNCTION h(k, m)     //k is key, m is table size
    hash_value ← k MOD m
    RETURN hash_value
ENDFUNCTION
```

Example: Insert these items in key-value format to a hash table of size 11:
(1,23), (2,38), (4,30), (10,21), (99,12), (29,138).

Key	$h(k) = k \bmod 11$	Hash value	Data
1	1	1	(1,23)
2	2	2	(2,38)
4	4	4	(4,30)
10	10	10	(10,21)
99	0	0	(99,12)
29	7	7	(29,138)

Table 1: Calculation of hash value

The hash table array will store data according to the hash value:

0	1	2	3	4	5	6	7	8	9	10
(99,12)	(1,23)	(2,38)		(4,30)			(29,138)			(10,21)

2.2 Hashing Character Data

When the key is an alphanumeric string, we must first convert it into a numeric form. One approach is to:

- Convert each character in the key to its **ASCII code**.
- Add all the ASCII values.
- Use modular hashing on the total.

```
FUNCTION hash_char(hash_key, m)        //hash_key is string, m is table size
    sum ← 0
    FOR i ← 0 TO LEN(hash_key) - 1     //LEN returns max size of string
        ascii_code ← CHAR_TO_CODE(hash_key[i])    //CHAR_TO_CODE
        sum ← sum + ascii_code         //returns the ASCII value of  parameter
    ENDFOR
    hash_value ← sum MOD m
    RETURN hash_value
ENDFUNCTION
```

Example: Assuming **size** of hash table **m** is **97**, and hash_key is **A5RD**.
ASCII values for characters A is 65, 5 is 53, R is 82, and D is 68.
Total of ASCII values is (65+53+82+68) = **268**.
The hash_value is given by **268** mod **97** = 74.
So, the key "**A5RD**" will be stored at index **74** in the hash table.

3. COLLISION

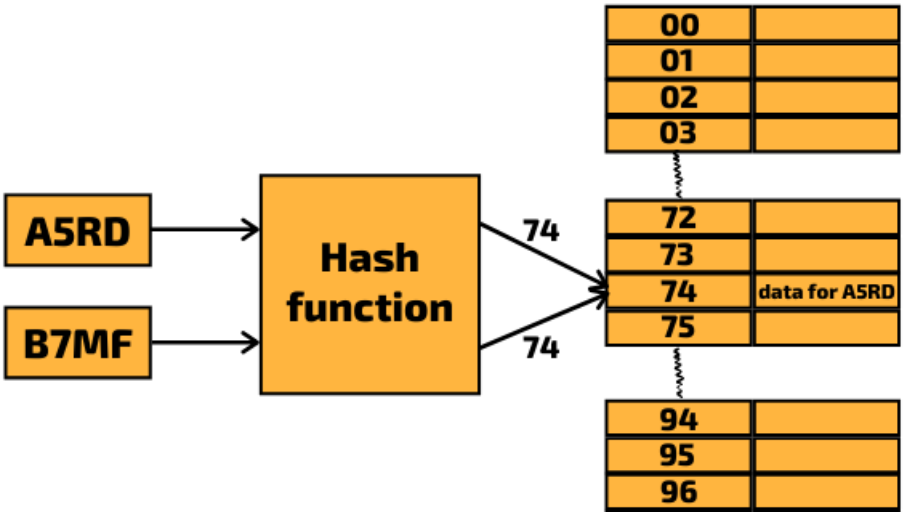
In practice, it is common for a **hash function to produce the same hash value for more than one key**. This situation is called a **collision**. **Collision** occurs when two or more different keys hash to the same index in the hash table.
If the hash function were perfect, it would assign a **unique index** for each key, ensuring no collisions. However, **perfect hash functions are rare**, especially when the number of possible keys is large compared to the size of the array.
Since **two different key-value pairs cannot occupy the same index in the array**, collisions must be handled using a suitable **collision resolution technique**

Example: Suppose the items with key-values `(15,321)` and `(32,32)` are added to the hash table shown in Table 1 above. The keys of these 2 items will be hashed to indices of the array that have already been occupied.

Key	h(k) = k mod *11*	Hash value	Data	
1	1	1	(1,23)	
2	2	2	(2,38)	
4	4	4	(4,30)	
10	10	10	(10,21)	
99	0	0	(99,12)	
29	7	7	(29,138)	
15	4	4	(15,321)	**collision!**
32	10	10	(32,32)	**collision!**

Table 1 with additional items `(15,321)` and `(32,32)`

Another example: Hashing of alphanumeric keys **A5RD** and **B7MF**, with size of hash table as 97. Both keys will be hashed to 74 and collision occurs.



Quiz

Fill in the hash values and indicate if collision occurs when the following key/value pairs get stored in an array of size 6 using this hash function.

```
FUNCTION hash_it(hash_key, array_size)
   hash_value ← hash_key MOD array_size
   RETURN hash_value
ENDFUNCTION
```

Key/value pair	Hash values	Collision occurs?
(12345, "Joyce")	3	
(28400, "Wendi")	2	
(17720, "Joyce")	2	Yes
(23423, "Nancy")	5	
(34238, "Paige")	2	Yes

Array of size 6

[0]	
[1]	
[2]	
[3]	
[4]	
[5]	

4. COLLISION HANDLING

In hash tables, a **collision** occurs when two or more keys are hashed to the same index. Collisions must be handled to ensure that data is stored and retrieved correctly. One common approach is **open addressing**, which includes methods such as **linear probing**.

4.1 Open Addressing – Linear Probing

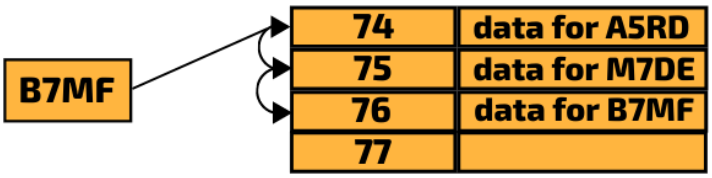
Linear probing is a collision resolution strategy where, if the desired slot is already occupied, the algorithm checks the next slot in a sequential manner until it finds an empty one. The search can also be **probed using an interval** (e.g. check every 3rd slot) **until an empty slot is found**.

It is important to note that if the key is not found at the index position generated by the hash function, it does not mean that it is not in the hash table. It could be found in a different position based on the probing sequence, and the algorithm will keep searching until an empty slot is encountered or until it completes the full cycle of the probing sequence without finding the key.

The following hash keys are hashed based on the size (97) of the hash table:

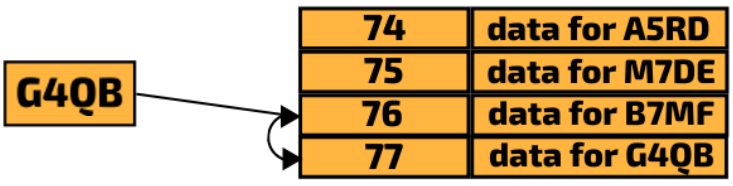
hash_key		Hash value
A5RD	hash_char(hash_key, 97)	74
M7DE		75
B7MF		74
G4QB		76

The data with the key **A5RD** has already been inserted into location 74 of the hash table. When an attempt is made to insert **B7MF**, the same hash value 74 is produced.



As location 74 is already occupied, the data with the key **B7MF** will be placed in the next free slot in the hash table. The **interval is 1**, so the next slot (index 75) will be examined. This is already occupied, so the next slot (index 76) is examined. This is empty so the data will be placed in this location.

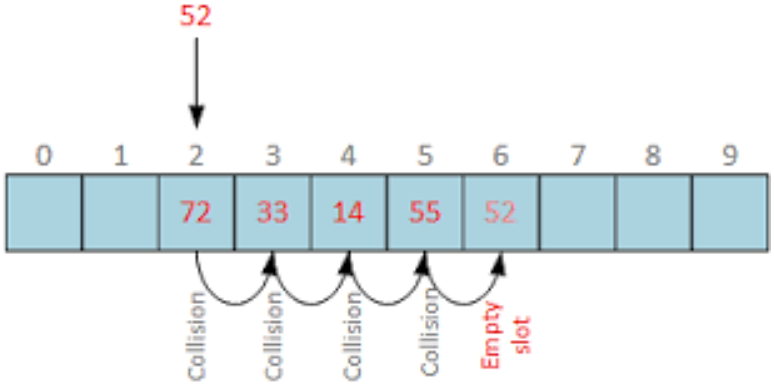
If there are a lot of collisions, the hash table can end up very messy. In the diagram below, notice how **G4QB**, which hashes to 76 has been placed at location 77. This is because location 76 has been occupied by the data for **B7MF** as a result of the earlier collision.



Clustering

Clustering is the situation where consecutive slots get filled due to collisions, forming a group (cluster). This slows down the insertion and search operations as longer sequences may need to be checked.

Example: Item 52 is hashed to location 2, which is already occupied and collision occurs. Next location 3 is probed but is occupied too. Collision occurs again. Repeating probing of next locations will be performed until an empty slot is found at location 6 to store 52.



There must be enough slots to store the volume of data. For example, if a database has 1 million records, then the size of hash table should be at least 1 million locations.

Load factor

The **load factor** of a hash table is a measure of how full the table is:

$$Load\ Factor = \frac{Number\ of\ Items\ Stored}{Size\ of\ Hash\ Table}$$

A **high load factor** increases the chance of collisions and slows down insertion and searching. To maintain efficiency, hash tables are often resized when the load factor exceeds a threshold (typically 0.7 to 0.8).

Searching

Searching for data from a hash table using linear probing:

- Hash the key to find the initial index.
- If the key at the index matches, return the data.
- If not, continue checking each subsequent slot (linear probing), wrapping to the beginning of the table if necessary, until either:
    - A matching key is found.
    - An empty slot is found, indicating the key is not present.

In the **worst-case scenario**, every position in the array may need to be inspected, resulting in a time complexity of **O(n)**. Therefore, it is important to maintain **extra capacity** in the hash table to reduce collision and ensure efficient searching.

Deleting

Deleting a record from an open hash table is problematic because setting the slot to NULL can disrupt subsequent searches. Instead, a common solution is to mark the slot with a **tombstone** – a special marker **indicating that the data has been deleted**. This preserves the probing sequence for search and insertion operations.

4.2 Open Addressing – Quadratic Probing

Quadratic probing is a collision resolution technique similar to linear probing but uses uses a different probing sequence. While linear probing checks the next slot by a constant step (e.g., +1, +2, +3), quadratic probing uses a quadratic function of the step number (e.g., $+1^2$, $+2^2$, $+3^2$), resulting in probe distances of 1, 4, 9, 16, and so on.

This reduces **primary clustering**, a problem in linear probing where long runs of filled slots increase the chance of further collisions. Quadratic probing spreads elements more widely across the table, making such clusters less likely.

Example:
Suppose we have a hash table of size 11 and the hash function is:

```
h(key) = key % 11
```

Insert the key 27: 27 % 11 = 5 → index 5 is empty → insert at index 5.
Now insert the key 38: 38 % 11 = 5 → index 5 is occupied → collision!

Use **quadratic probing**:

- Try index $5 + 1^2 = 6$ → if empty, insert.
- If not, try $5 + 2^2 = 9$
- Then $5 + 3^2 = 14$ → $14 \% 11 = 3$
- Continue with $5 + 4^2 = 21$ → $21 \% 11 = 10$, etc.

Insert 38 at the first empty index found using this sequence.

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Contents						**27**	**38**				

(Here, 27 was inserted at index 5, and 38 at index 6 using $+1^2$)

4.3 Separate Chaining (Close Addressing)

In separate chaining, collisions are handled using **linked lists**. Each key is mapped to a **fixed address** (close addressing) in the hash table, which stores a **pointer to the head of a linked list** rather than the data itself. Each node in the list contains:

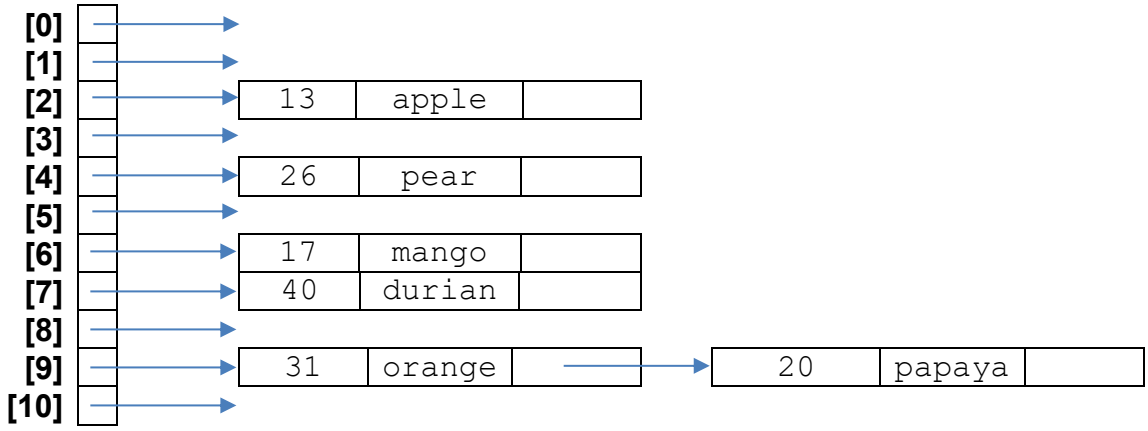
- the **key**
- the **data**
- a **pointer to the next node**

When a new item is added:

- If the index is empty, a node is created with its next pointer set to **NULL**.
- If a collision occurs (another key hashes to the same index), a new node is inserted at the **head** of the list, and its pointer links to the previous first node.

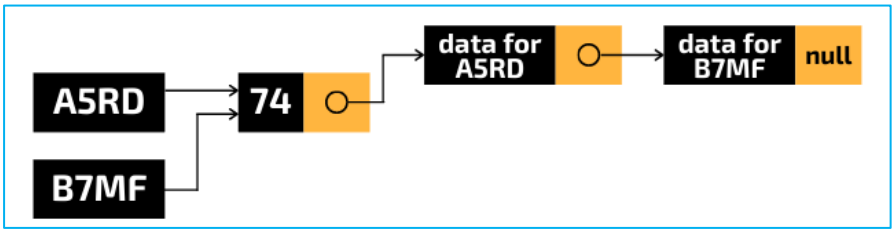
This forms a **chain of nodes** at the same index, all sharing the same hash value.

Example: A hash table of 11 locations with chains of nodes. The key-value pair are first hashed to the indexed location and then added to the linked list at that location. The key 13 is hashed to index 2 and data item (13, apple) is stored in a node at that location. The keys 20 and 31 are both hashed to index 9 and stored in a chain of nodes.



Another example: When data with keys **A5RD**, **B7MF** are added to a closed hash table, the data with the key **A5RD** is stored as a **linked list node**, where **a pointer to that node is stored in the hash table at location 74**.

No	Key	Hash Function h	Hash value
1	A5RD	h(A5RD)	74
2	B7MF	h(B7MF)	74



When collisions are handled using separate chaining, data is always accessed through a single hash table index.

The **searching** process is as follows:

- Generate the hash key to find the index value.
- Follow the pointer at that index to the head of the linked list.
- If the head is empty, the item is not found.
- Otherwise, examine each node in the list until the key is found or the list ends.

Deleting an item involve first searching for it. Once found, the node is removed similarly to deleting from a linked list.

In the worst-case scenario (all keys hash to the same value), searching degrades to a linear search on a linked list with time complexity **O(n)**. Thus, designing a good hash function to minimise collisions is crucial.

4.4 Comparing Linear Probing and Separate Chaining

No.	Linear Probing	Separate Chaining
1.	Hash table may become full, especially as the load factor increases. Since the array size is fixed, resizing and rehashing operations may be needed to accommodate more elements efficiently.	Hash table can grow indefinitely by adding new nodes to linked lists. Resizing the hash table is not needed as long as there is enough memory space to allocate new nodes.
2.	Requires marking deleted locations as 'deleted' using tombstone markers.	Deleted nodes are removed from the linked list and freed; no tombstone markers needed.
3.	Mostly use when the number of keys is known in advance and the table can be kept sparsely filled, as performance drops due to clustering at high load factors.	Mostly used when the number of keys is unknown or when the hash table must handle a variable number of elements without resizing.
4.	Can suffer from clustering, especially at high load factors, as consecutive keys with the same hash index create clusters of occupied slots.	No clustering occurs because keys that hash to the same index are stored in separate linked lists, keeping data spread out evenly.
5.	Better cache performance since elements are stored in contiguous memory locations within the array.	Cache performance may be lower since linked list nodes are not stored contiguously, and traversal is required.

5. TUTORIAL

1. [RVHS/2018/Prelim/9597/P2/Q1a]
A hash table has an index range of 1 to 11. The following pseudocode describes an algorithm which uses the hash function, Hash(Key) = Key mod 11 with linear probing to store a key in the hash table.

Line number

```
1       Index ← Hash(Key)
2       WHILE Table[Index] <> Key
3           Index ← Index + 1
4       ENDWHILE
5       Table[Index] ← Key
```

Identify **3 issues** in the algorithm and suggest modifications to overcome them.

2. [NJC/2019/Prelim/9597/P2/Q4b]
When implementing a Hash Table, it is typically necessary to ensure that your implementation also includes collision resolution. Two categories of collision resolution mechanisms for Hash Tables are Open Addressing and Separate Chaining.
(a) Describe the algorithm for the opening addressing mechanism of quadratic probing.
(b) Describe the difference between linear probing and quadratic probing, and explain why one might wish to use quadratic probing instead of linear probing.
(c) A computer scientist has implemented the following `INSERT` function for a Hash Table.

```
FUNCTION INSERT(hashTable: ARRAY OF OBJECT, data: OBJECT)
RETURNS ARRAY OF OBJECT
    DECLARE hashValue, step: INTEGER
    IF ISFULL(hashTable) THEN
        OUTPUT "Hash Table full. Unable to insert."
        RETURN NULL
    ENDIF
    hashValue ← HASH(data) % hashTable.SIZE()
    step ← -1
    WHILE hashTable[hashValue] <> NULL DO
        hashValue ← hashValue + step
        IF hashValue < 1 THEN
            hashValue ← hashTable.SIZE()
        ELSE IF hashValue > hashTable.SIZE() THEN
            hashValue ← 1
        ENDIF
        step ← step * (-2)
    ENDWHILE
    hashTable[current] ← data
    RETURN hashTable
ENDFUNCTION
```

Determine if the collision resolution method utilised in the `INSERT` function defined above is valid. Justify your answer by using a counter example.

6. PROGRAMMING EXERCISE

1. Consider a customer ID of alphanumeric key, length 4. The hash function h will first convert each character of the alphanumeric key to its ASCII number code, and have them all summed up to get a total. Finally, the hash key is obtained by applying modulo-*N* to the total, where *N* is the capacity of the hash table.

   (a) Given that the size of hash table is 9, implement the hash function in Python and compute the hash value in following table:

key	h(key)
A5RD	
B7MF	
R2YJ	
J6TR	
G4QB	

`Customer_ID` and `Customer_name` will be stored in an array in the hash table.

   (b) Write program `CreateHashTable` to create and return a hash table that has a capacity of size 9, implemented by an array of arrays, with the fields `Customer_ID` and `Customer_name` initialised to empty strings.

index	value
0	[" ", " "]
1	[" ", " "]
2	[" ", " "]
…	….
8	[" ", " "]

Hash Table

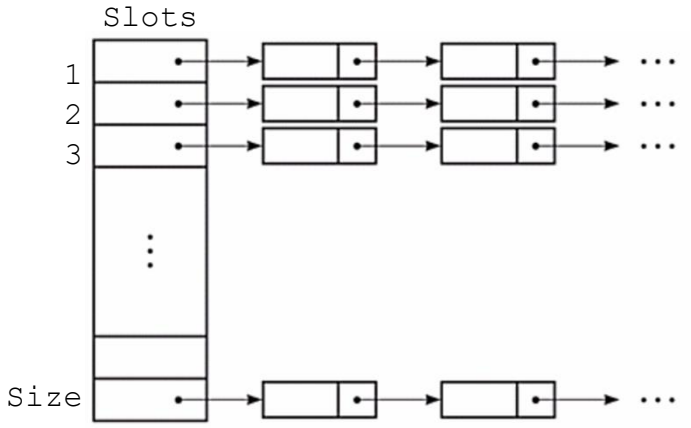
   (c) Using linear probing, write program `AddRecord(filename)` that will read data from "`customers.txt`" and insert the record of each customer into the hash table. The file stores sample data as shown: `(create your own text file)`

```
A5RD, Carl Chen
B7MF, Peter Poh
R2YJ, Marcus Mak
J6TR, Nancy Nah
G4QB, Elizabeth Ee
```

   (d) Write program `Search(Customer_ID)` that returns the address of the record in the hash table if found and -1 if not found.

   (e) Write program `Delete(Customer_ID)` that will delete the record of a customer using `Customer_ID`.

2. The task is to store records of books in a hash table and have direct access to records. Design the hash table as an array of linked list to handle collision of records. A hash function mapped into a location in the array, which contains a pointer to a linked list of records. When a collision occurs, records with the same hash code will be chained together.



Each node in the linked list contains a record of a book and implemented as an instance of the class `BookRec`.

The class `BookRec` has the following properties:

Class: `BookRec`	
Properties	
Identifier	**Description**
`BookID`	ID of a book
`Title`	Title of a book
`Pointer`	Pointer of node

(a) Write program code for the class `BookRec`, using suitable data types for the attributes.

A linked list is implemented as an instance of the class `LinkedList`. The class `LinkedList` has the following properties and methods:

Class: `LinkedList`	
Properties	
Identifier	**Description**
`Start`	Points to the first node of the linked list
Methods	
`Initialise`	Initialises `Start` to be Null.
`IsEmpty`	Tests for empty linked list. Returns Boolean.
`AddNode`	Adds a new node to the linked list. Takes `BookID` and `Title` as parameters.
`DeleteNode`	Deletes a node from the linked list. Takes `BookID` as parameter.
`SearchNode`	Searches for an existence of a record. Takes `BookID` as parameter.
`DisplayLinkedList`	Displays current state of pointers and contents of linked list in a suitable and clear format.

(b) Write program code for the class `LinkedList`.

The hash table is implemented as an instance of the class `HashTable`. The class `HashTable` has the following properties and methods:

Class: `HashTable`	
Properties	
Identifier	**Description**
`Size`	Size of the array
`Slots`	Array of linked list that makes up the hash table
Methods	
`Initialise`	Initialises `Size` and `Slots`.
`Hash`	A hashing function that calculates the address of the hash table
`Display`	Displays the contents of the hash table.
`Put`	Puts a record into the hash table after calculating the address to store the record. Takes `BookID` and `Title` as parameters.
`Remove`	Removes a record from the hash table. Takes `BookID` as paramter.
`Search`	Searches for a record in the hash table. Takes `BookID` as parameter. Returns Boolean.

The address in the hash table is calculated from a **hashing function** as follows:

- The ASCII code is calculated for each character within the `BookID`.
- The total of all ASCII values is calculated.
- The total is divided by `Size` and the remainder calculated with modulo arithmetic, where `Size` is the maximum address location.
- The value returned by the function is the (remainder + 1). This value is the address for the record in the hash table.

For example, for a `Size` of value 17, if the book record has `BookID` **CS733,** the value from the hashing function is 2. Therefore, write record with `BookID` **CS733** to the array with address 2.

(c) Write program code for the class `HashTable`, including the `Hash` function which uses the following specifications:

```
FUNCTION Hash (BookID: STRING) RETURNS INTEGER
```

(d) Test your program code for `Size` of value 17, by

(i) putting the following records into the hash table and display the hash table.

BookID	**Title**
CS733	Basic algorithms
AB944	Master Computing
KS293	Data structures
BK232	Programming exercises
PK199	Testing Python

(ii) removing AB944 from the hash table and display the hash table.

(iii) searching for KS293 from the hash table.

7. APPENDIX

Other hashing functions

There are many other common hashing functions besides the modular hashing. For example, preconditioning the key, mid-square hashing and folding method.

Preconditioning The Key

If the key is a string, we first convert the alphabetic key to a numeric key. Then using the modular hashing function method, generate an address location for the key.

Example: Let 'A' be 11, 'B' be 12 and so on, the string *Peter* will be converted to the numeric key 2615301528. If there are 25 locations, the hashing value of the string *Peter* will be 3.

Mid-Square Hashing Method

In this method, the key value is squared and the address of the record can be obtained by extracting some digits from the middle of the square.

Key	Square	4-Digit Address	3-Digit Address
123456	15241383936	1383	138
113586	12901779396	1779	177

Folding Method

In this method, a key is partitioned into parts. Each part is of the same length as the required address. These parts are then added to give the address.

Key	3-Digit Address	4-Digit Address
123456	123 + 456 = 579	1234 + 56 = 1290
187249653	187 + 249 + 653 = 089	1872 + 4965 + 3 = 6840